1. Branch predictor implementation

Ali Alfageeh
October 15, 2026

This part of the assignment implements the branch-direction predictor in `pm_predictor`. It combines a bimodal table with a tagged global predictor. The global prediction is used when its tag matches; otherwise, the bimodal table supplies the answer. Both tables learn from the actual branch outcome.

The implementation uses the two-way global table allowed by the handout. Loop prediction, branch-target prediction and the Pentium M path information register are left out, as permitted. The result is the required subset of the predictor rather than a model of the complete Pentium M branch unit.

Main result

Across the 20 supplied traces, the hybrid predictor averages **7.260 mispredictions per 1,000 instructions (MPKI)**. The bimodal predictor averages 10.299 MPKI. The hybrid therefore improves on bimodal overall, although the supplied, larger gshare predictor performs better at 6.306 MPKI.

Predictor	Direction errors	Average MPKI
Always taken	172,623,375	86.312
Bimodal	20,597,217	10.299
Supplied gshare	12,611,718	6.306
Implemented hybrid	14,519,207	7.260

Files changed and added

The predictor is in `cbp2-infrastructure-v2/src/my_predictor.h`. The original driver and trace decoder are retained. The decoder's command pointer was made const so that it can hold string literals; the decoding algorithm was not changed.

The added predictor tests check state updates and replacement. A separate evaluator runs four predictors on the same trace, and `scripts/benchmark.py` collects their results and checks the hybrid against the original driver. This keeps the comparison and the required submission driver tied to the same implementation.

2. Predictor organization

State	Configuration
Bimodal table	4,096 two-bit counters; address bits [11:0] select a counter
Global table	512 sets, two ways per set; valid bit, six-bit tag and two-bit counter
History	15 most recent conditional outcomes; initially zero
Replacement	One LRU victim bit per set
Initial state	Bimodal counters are 1 (weakly not taken); global entries are invalid

The address/history lookup uses the following expressions. The XOR is a simple gshare-style choice for the hash that the assignment allows us to simplify.

```
bimodal_index = address & 0xFFF
hash          = (address ^ history) & 0x7FFF
global_set    = hash >> 6
global_tag    = hash & 0x3F
```

A valid matching global tag overrides the bimodal prediction. A counter of 0 or 1 predicts not taken; 2 or 3 predicts taken. The lookup saves the selected indices, tag and hit way before any history update, so training changes the entry that made the prediction.

After a conditional branch resolves, its bimodal counter is trained. A global hit trains the matching counter. A miss allocates an invalid way if one exists, or replaces the LRU way otherwise. The new counter is 2 after taken and 1 after not taken. The accessed way becomes most recently used, then the actual outcome is shifted into the 15-bit history.

A small check on the indexing

For address 0x44 and history 0, the bimodal index is 68, the global set is 1 and the tag is 4. Both global ways are initially invalid, so the cold prediction is not taken. If the branch is taken, the bimodal counter becomes 2, a global entry with counter 2 is allocated, and history becomes 1. The next lookup uses that new history.

Unconditional branches predict taken and do not train these direction tables. The logical storage is 8,192 bimodal bits + 9,216 global-entry bits + 512 replacement bits + 15 history bits = **17,935 bits**, or about 2.19 KiB. C++ storage is larger because counters and fields occupy bytes and the simulator keeps an update record.

3. Tests and measurement method

Seven deterministic test groups exercise the parts most likely to fail even when a predictor produces a plausible overall MPKI. The optimized test target passes all seven.

Test group	What it checks
Cold state	Invalid entries, zero tags, fallback, and lookup without training
Indexing	Hand-calculated hashes, large addresses and byte-address differences
Counters and selection	All two-bit transitions, saturation, both ways and global priority
Replacement	Invalid-way preference, LRU refresh, eviction and new-tag initialization
Saved lookup	Training uses the old index before history shifts
Unconditional branches	Direction state is unchanged for jumps, calls and returns
Learning patterns	Repeated taken, repeated not-taken and alternating outcomes

Trace experiment

Each of the 20 course traces represents 100 million instructions. The full experiment therefore covers 2 billion instructions and 275,461,634 conditional branches. Each trace starts with fresh predictor state, including cold training; there is no discarded warm-up.

The evaluator compares always-taken, a 4,096-entry bimodal table, the supplied 32,768-entry gshare and the hybrid on the same decoded outcomes. The hybrid and bimodal counters start weakly not taken. The supplied gshare retains its original strongly not-taken initialization.

```
MPKI = conditional direction errors * 1000 / instructions
```

This denominator counts instructions, not branches. The aggregate MPKI is also the arithmetic mean of the 20 trace values because all traces have equal instruction counts. The global hit rate is weighted by the number of conditional branches.

The saved experiment used macOS/arm64 and Apple Clang 17.0.0 with the supplied -g -O3 -Wall flags. The benchmark script checked each hybrid MPKI against the original driver. For this report, the saved counts and arithmetic were checked again, along with all five recorded source checksums and all 20 local trace checksums. The full trace experiment was not rerun just to format the PDF.

4. Results on all 20 traces

All error columns are MPKI; lower is better. The last column is the fraction of conditional predictions that found a matching global entry.

Trace	Always taken	Bimodal	Gshare	Hybrid	Global hit
164.gzip	70.025	15.030	12.473	12.617	83.06%
175.vpr	85.312	14.671	13.415	14.337	56.09%
176.gcc	86.071	19.347	11.254	15.298	54.43%
181.mcf	102.497	33.245	15.837	17.680	77.42%
186.crafty	48.651	13.075	5.837	8.464	56.55%
197.parser	78.421	15.417	10.008	10.357	80.06%
201.compress	76.983	9.356	7.831	8.199	87.68%
202.jess	92.343	8.809	1.562	1.750	92.36%
205.raytrace	109.506	3.806	2.756	3.421	62.95%
209.db	91.451	6.480	3.909	4.613	86.70%
213.javac	109.644	3.181	2.267	2.638	60.98%
222.mpegaudio	110.339	3.154	2.188	2.604	61.23%
227.mtrt	112.022	3.706	2.657	3.268	66.69%
228.jack	77.958	7.691	3.033	3.946	75.30%
252.eon	24.226	9.405	1.807	3.139	76.86%
253.perlbmk	66.833	5.955	2.554	4.700	72.93%
254.gap	75.950	9.675	3.926	4.797	87.01%
255.vortex	70.524	1.916	1.222	1.239	83.95%
256.bzip2	178.257	0.112	0.094	0.076	99.62%
300.twolf	59.221	21.942	21.489	22.046	32.68%
Average / total	86.312	10.299	6.306	7.260	74.02%

Source: results/benchmark.csv. Values are rounded only for display. The CSV retains miss counts, instruction counts, hit counts and trace checksums.

The hybrid supplies 203,900,607 predictions from the global table, a hit rate of 74.02%. A hit only means a matching tag was found; it does not mean the prediction was correct. The bimodal table supplies the remaining predictions.

5. Discussion and limitations

The hybrid reduces total direction errors by **29.51% compared with bimodal** and improves on it in 19 of the 20 traces. History is particularly useful on 202.jess, where MPKI falls from 8.809 to 1.750. On 181.mcf it falls from 33.245 to 17.680. These results show the value of recording the context of a branch instead of relying only on its address.

The exception is 300.twolf. Its hybrid MPKI is 22.046, slightly worse than bimodal at 21.942. Only 32.68% of its conditional lookups hit in the global table, and a global hit always overrides the bimodal answer. The comparison shows a regression, but it does not isolate whether aliasing, replacement or that selection rule caused it.

Comparison with the supplied gshare

Gshare averages 6.306 MPKI, compared with 7.260 for the hybrid. The hybrid beats gshare only on 256.bzip2, where the values are 0.076 and 0.094. It would be misleading to use this experiment to conclude that the hybrid organization is always worse: the supplied gshare has 32,768 counters, while the hybrid global table has only 1,024.

The supplied gshare uses 65,551 logical bits including its history, about 8 KiB. The hybrid uses about 2.19 KiB. The predictors also start with different counter states. A comparison at equal storage and equal initialization would be needed to separate the effect of organization from capacity and cold-start behavior.

What works and what is outside this submission

The required bimodal/global lookup, two-bit training, tag checks, two-way replacement and conditional history updates are implemented. The deterministic tests pass, and the saved trace measurements agree with the assignment driver. There is no known failing case in the tests described here; those tests do not prove correctness for every possible trace.

The optional full Pentium M extra-credit class remains a starter skeleton. It is not used for the reported results. Target prediction, the loop predictor and PIR are omitted under the handout's allowed scope. The report also makes no claims about processor IPC, power or the timing cost of the predictor.

The main outcome is a working compact hybrid predictor with lower error rates than bimodal on most of these traces. Its weaker results against gshare and the small regression on twolf remain part of the result, rather than being excluded from the average.

6. Reproducing the results and sources

From the repository root, enter the BP folder and build the required driver and tests. A C++ compiler, make, Python 3.8+ and the decoder's gzip/bzip2 tools are needed.

```
cd Assignment1/BP
make -C cbp2-infrastructure-v2/src all test
```

Copy the course trace directory into cbp2-infrastructure-v2/traces, then run:

```
python3 scripts/benchmark.py
cd cbp2-infrastructure-v2
./src/predict traces/164.gzip/gzip.trace.bz2
```

The single-trace command prints 12.617 MPKI. The comparison script regenerates benchmark.csv, benchmark-summary.md, driver-results.txt and environment.txt in results/. The original csh runner is also supported. It truncates the mean to 7.259; rounding the exact aggregate gives 7.260.

Rebuild both PDF reports

Run the following from the repository root. ReportLab is only needed for PDF creation.

```
python3 -m venv .report-venv
.report-venv/bin/pip install -r requirements-report.txt
.report-venv/bin/python scripts/build_assignment_reports.py
```

Sources and attribution

1. *Assignment 1: Branch Predictor Implementation*, supplied course handout, especially pp. 1-3 and Appendix B. The required subset and two-way option come from this handout.

2. Daniel A. Jimenez, supplied branch-prediction competition infrastructure. The trace reader, driver, gshare example, original runner and documentation were supplied with the course. See cbp2-infrastructure-v2/doc/index.html.

3. Vladimir Uzelac and Aleksandar Milenkovic, *Experiment Flows and Microbenchmarks for Reverse Engineering of Branch Predictor Structures*, ISPASS 2009, pp. 207-217, DOI: 10.1109/ISPASS.2009.4919652. This is the background reference identified in the handout; the implementation decisions reported here are checked against the submitted source.

No external cache implementation was used. Codex assisted with code, tests and report preparation. Numerical tables are based on the recorded program outputs.

Repository: fqeh/CompArchAssignment1 / Assignment1/BP